Pension Fund Efficiency Toolkit

Technical Explanation: Calculations, Data, Graphs, and Process Tests

Zimbabwe Occupational Pension Funds (2018–2022)

Pension Efficiency Research Project

March 2026

Contents

1 Overview: What We Are Doing and Why **3**

2 Dataset: Column-by-Column Reference **3**

3 Stage 1A: PCA Input Validation **5**
3.1 Purpose . . . 5
3.2 Calculation . . . 5
3.3 Scree plot interpretation . . . 6

4 Stage 1B: CCR (CRS) DEA Model **6**
4.1 Background . . . 6
4.2 Notation . . . 6
4.3 Two-Phase LP Formulation . . . 7
4.4 Why two phases? . . . 7
4.5 Output file: `out/efficiency_ccr.csv` . . . 7

5 Stage 1C: BCC (VRS) DEA Model **8**
5.1 Background . . . 8
5.2 Modification to Phase I . . . 8
5.3 Output file: `out/efficiency_vrs.csv` . . . 8

6 Stage 1D: Scale Efficiency and Returns to Scale **8**
6.1 Scale efficiency . . . 8
6.2 Returns to Scale classification . . . 8
6.3 Output file: `out/scale.csv` . . . 9

7 Stage 1E: Peer Analysis and Input Reduction Targets **9**
7.1 Target inputs . . . 9
7.2 Reduction metrics . . . 9
7.3 Output file: `out/targets.csv` . . . 9

8 Stage 2A: Simar–Wilson Bootstrap Bias Correction **9**

8.1 Why bias correction is needed . . . 9
8.2 Algorithm . . . 9
8.3 Output file: `out/bias_corrected_scores.csv` . . . 10

9 Stage 2B: Random Forest Second Stage **10**
9.1 Purpose . . . 10
9.2 Model . . . 10
9.3 Feature importance measures . . . 11
9.4 Partial Dependence Plots . . . 11
9.5 Output files . . . 11

10 Graph Descriptions **11**
10.1 PCA Scree Plot . . . 11
10.2 CCR vs. BCC Efficiency Bar Chart . . . 11
10.3 Partial Dependence Plots . . . 11

11 Process Verification: Python Test Snippets **12**
11.1 Stage 0: Data loading and schema . . . 12
11.2 Stage 1: PCA variance explained . . . 12
11.3 Stage 2: CCR DEA correctness . . . 13
11.4 Stage 3: BCC $\geq$ CCR and Scale Efficiency in (0, 1] . . . 14
11.5 Stage 4: Input reduction targets are valid . . . 15
11.6 Stage 5: Bootstrap bias correction properties . . . 16
11.7 Stage 6: Random Forest second stage . . . 17
11.8 Stage 7: End-to-end pipeline sanity check . . . 18

12 Running the Full Pipeline **19**

13 Key References **20**

1 Overview: What We Are Doing and Why

Zimbabwe's pension sector manages the retirement savings of hundreds of thousands of workers across public and private employment. The sector is regulated by the Insurance and Pensions Commission (IPEC) and operates under extreme macroeconomic stress: hyperinflation peaked above 500% in 2020, multiple currency reforms were enacted between 2018 and 2022, and COVID-19 disrupted member contributions and investment markets simultaneously.

The central question of this study is: *Among Zimbabwean occupational pension funds, which funds convert their resources into member benefits most efficiently, and what structural or environmental factors explain differences in efficiency?*

We answer this question using a two-stage empirical framework:

1. **Stage 1 — Efficiency measurement (Data Envelopment Analysis).** DEA is a non-parametric frontier method that assigns each Decision-Making Unit (DMU, here: each pension fund) an efficiency score $\theta \in (0, 1]$ by comparing it against the best-practice frontier formed by the data itself. A score of 1.0 means the fund is on the frontier (fully efficient); a score of 0.7 means it could achieve the same outputs with only 70% of its current inputs.
2. **Stage 2 — Determinant analysis (Random Forest regression).** Once bias-corrected efficiency scores are obtained, we regress them on environmental variables (inflation, exchange rate volatility, fund age) to identify which contextual factors drive efficiency differences.

The pipeline also performs:

- **PCA** on the input variables to check multicollinearity and optionally reduce dimensionality before DEA.
- **Simar–Wilson bootstrap** to correct the upward bias inherent in DEA estimators and to construct confidence intervals around each score.
- **Scale efficiency analysis** to classify each fund as operating under Increasing, Decreasing, or Constant Returns to Scale.
- **Peer analysis** to identify which frontier funds serve as benchmarks for each inefficient fund and to compute concrete input-reduction targets.

2 Dataset: Column-by-Column Reference

The analysis uses a panel dataset of 20 Zimbabwean pension funds observed annually from 2018 to 2022 (100 rows total). The file is `tests/dataset.csv`.

Column	Type	Description
`fund_id`	string	Unique fund identifier (`ZW001`–`ZW020`). Stable across years.
`year`	integer	Observation year: 2018, 2019, 2020, 2021, or 2022.

Column	Type	Description
`fund_name`	string	Full name of the fund (e.g. *NationalSocialSecurityAuthority*). Constant within a fund.
`fund_type`	string	`public` (government-sponsored) or `private` (employer-sponsored occupational scheme).
`total_assets_usd`	float	Total assets under management expressed in USD equivalent at year-end. **DEA input.**
`operating_expenses_usd`	float	Annual administrative and management expenses in USD. Includes staff costs, custodian fees, and audit fees. **DEA input.**
`equity_debt_usd`	float	Market value of equity plus fixed-income portfolio in USD. Proxy for the fund's investable asset base. **DEA input.**
`net_investment_income_usd`	float	Total investment returns less investment management costs in USD. Can be negative during crisis years (2019–2020). **DEA output.**
`member_contributions_usd`	float	Total contributions received from members and employers during the year in USD. **DEA output.**
`inflation`	float	Annual CPI inflation rate for Zimbabwe (%). Sourced from ZIMSTAT. **Environmental variable (Stage 2).**
`exchange_volatility`	float	Annualised standard deviation of the ZWL/USD exchange rate, normalised to $[0, 1]$. Higher values indicate greater currency instability. **Environmental variable (Stage 2).**
`fund_age`	integer	Years since the fund's inception as of the observation year. Increments by 1 each year. **Environmental variable (Stage 2).**

Macroeconomic variables by year

The values below are fixed per year and identical for all funds within a year, reflecting the shared macroeconomic environment.

Year	Inflation (%)	Exchange Volatility
2018	10.6	0.18
2019	255.3	0.72
2020	557.2	0.85
2021	98.5	0.45
2022	243.8	0.61

Table 2: Zimbabwe macroeconomic environment by year.

Why these inputs and outputs?

- **Inputs** represent resources consumed: assets deployed, expenses incurred, and capital invested. An efficient fund achieves maximum output from a given resource bundle.
- **Outputs** represent value delivered to members: investment returns and contribution flows.
- **Environmental variables** are not under the fund manager's control (inflation, exchange rate) or only slowly controllable (fund age, size tier). They are excluded from the DEA frontier but analysed in Stage 2.

Cross-sectional vs. panel: Before running DEA, observations are aggregated to one row per fund by taking the mean across the five years. This gives a single *average-period* efficiency score per fund.

3 Stage 1A: PCA Input Validation

3.1 Purpose

Before running DEA we apply Principal Component Analysis (PCA) to the three input variables. The goals are:

1. Check for multicollinearity (if $\geq 95\%$ of variance is explained by a single component, the inputs are near-collinear and should be consolidated).
2. Optionally replace the three raw inputs with two PCA composites to satisfy the DEA rule of thumb $n \geq 3(m + s)$, where n is the number of DMUs, m the number of inputs, and s the number of outputs.

3.2 Calculation

Let $\mathbf{X} \in \mathbb{R}^{n \times m}$ be the input matrix ($n = 20$ funds, $m = 3$ inputs).

Step 1 — Standardisation. Compute the standardised matrix:

$$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

where $\bar{x}_j$ and s_j are the column mean and standard deviation. This ensures financial columns measured in different magnitudes contribute equally.

Step 2 — Covariance / correlation. The correlation matrix is:

$$\mathbf{C} = \frac{1}{n-1}\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$$

Step 3 — Eigendecomposition.

$$\mathbf{C} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\top$$

where $\boldsymbol{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_m)$ are eigenvalues in descending order and $\mathbf{V}$ contains the corresponding eigenvectors (principal component loadings).

Step 4 — Explained variance ratio.

$$\mathrm{EVR}_k = \frac{\lambda_k}{\sum_{j=1}^{m} \lambda_j}$$

Step 5 — Transform. The principal component scores:

$$\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{V}_{:,1:K} \quad (K \text{ components retained})$$

3.3 Scree plot interpretation

The scree plot bar chart shows $\mathrm{EVR}_k \times 100$ for each component. A steep drop after the first component ("elbow") indicates strong collinearity. In our data the first component typically explains $\geq 80\%$ of variance, confirming that the three input variables are highly correlated (all scale with fund size).

4 Stage 1B: CCR (CRS) DEA Model

4.1 Background

Data Envelopment Analysis was introduced by Charnes, Cooper and Rhodes (1978). The CCR model assumes Constant Returns to Scale (CRS): doubling all inputs exactly doubles all outputs. It measures *overall technical efficiency*.

4.2 Notation

- n = number of DMUs (funds), indexed $j = 1, \ldots, n$
- m = number of inputs, indexed $i = 1, \ldots, m$
- s = number of outputs, indexed $r = 1, \ldots, s$
- x_{ij} = input i of DMU j
- y_{rj} = output r of DMU j
- θ_k = efficiency score of DMU k under evaluation
- λ_j = intensity variable (contribution of DMU j to the reference set)
- s_i^- = input slack for input i
- s_r^+ = output slack for output r

4.3 Two-Phase LP Formulation

The CCR model for DMU k is solved as two sequential linear programs.

Phase I: Radial Efficiency Score

$$\min_{\theta,\boldsymbol{\lambda}} \quad \theta_k \tag{1}$$

$$\text{s.t.} \quad \sum_{j=1}^{n} \lambda_j x_{ij} \leq \theta_k x_{ik} \quad \forall\, i = 1, \ldots, m \tag{2}$$

$$\sum_{j=1}^{n} \lambda_j y_{rj} \geq y_{rk} \quad \forall\, r = 1, \ldots, s \tag{3}$$

$$\lambda_j \geq 0 \quad \forall\, j, \quad 0 \leq \theta_k \leq 1$$

The constraint (2) says the reference set must produce the evaluated fund's outputs using no more than θ_k times its inputs. The optimal θ_k^* is the *radial contraction factor*: if $\theta_k^* = 0.8$ the fund could shrink all inputs by 20% and still produce the same outputs.

Phase II: Slack Maximisation

With θ_k^* fixed, Phase II maximises the non-radial slacks:

$$\max_{\boldsymbol{\lambda},\mathbf{s}^-,\mathbf{s}^+} \quad \sum_{i=1}^{m} s_i^- + \sum_{r=1}^{s} s_r^+ \tag{4}$$

$$\text{s.t.} \quad \sum_{j=1}^{n} \lambda_j x_{ij} + s_i^- = \theta_k^* x_{ik} \quad \forall\, i \tag{5}$$

$$\sum_{j=1}^{n} \lambda_j y_{rj} - s_r^+ = y_{rk} \quad \forall\, r \tag{6}$$

$$\lambda_j, s_i^-, s_r^+ \geq 0 \tag{7}$$

A DMU is **fully CCR-efficient** if and only if $\theta_k^* = 1$ *and* all slacks are zero.

4.4 Why two phases?

The single-phase formulation $\min \theta - \varepsilon(\sum s_i^- + \sum s_r^+)$ is theoretically equivalent but numerically unstable when input values are in the hundreds of millions (USD). At such scales $\varepsilon \cdot x_{ik} \gg 1$, making the LP unbounded. The two-phase approach avoids this entirely.

4.5 Output file: `out/efficiency_ccr.csv`

Column	Meaning
`fund_id`	Fund identifier
`theta_ccr`	CCR efficiency score $\theta_k^* \in (0, 1]$
`slack_*`	Per-input slack s_i^- from Phase II
`peer_ids`	Pipe-separated IDs of reference set DMUs ($\lambda_j > 10^{-4}$)

5 Stage 1C: BCC (VRS) DEA Model

5.1 Background

The BCC model (Banker, Charnes, Cooper, 1984) adds a *convexity constraint* $\sum_j \lambda_j = 1$, relaxing the CRS assumption to allow Variable Returns to Scale. BCC measures *pure technical efficiency*, isolating managerial efficiency from scale effects.

5.2 Modification to Phase I

The only change to the CCR Phase I LP is the addition of:

$$\sum_{j=1}^{n} \lambda_j = 1$$

This forces the reference virtual DMU to lie on the convex hull of the observed data rather than on a ray through the origin.

Remark 1. *Because VRS is more permissive than CRS, BCC scores are always $\geq$ CCR scores for the same DMU: $\theta_k^{BCC} \geq \theta_k^{CCR}$.*

5.3 Output file: `out/efficiency_vrs.csv`

Same structure as `efficiency_ccr.csv` but the score column is `theta_bcc`.

6 Stage 1D: Scale Efficiency and Returns to Scale

6.1 Scale efficiency

Definition 1. *The **scale efficiency** of DMU k is:*

$$SE_k = \frac{\theta_k^{CCR}}{\theta_k^{BCC}}$$

$SE_k \in (0, 1]$. A value of 1 means the fund is operating at its most productive scale size (MPSS); values below 1 indicate scale inefficiency.

6.2 Returns to Scale classification

The nature of returns to scale is determined from the BCC optimal lambda vector $\boldsymbol{\lambda}^*$:

Condition	Classification	Interpretation
$\sum_j \lambda_j^* \approx 1$ and $SE_k \approx 1$	CRS	Optimal scale
$\sum_j \lambda_j^* < 1$	IRS	Fund is too small
$\sum_j \lambda_j^* > 1$	DRS	Fund is too large

Expected pattern in our data:

- Large public funds (NSSA, civil service) → DRS
- Small agriculture/university funds → IRS
- Medium efficient funds → CRS

6.3 Output file: `out/scale.csv`

Columns: `fund_id`, `scale_efficiency`, `rts_classification`.

7 Stage 1E: Peer Analysis and Input Reduction Targets

7.1 Target inputs

For each inefficient DMU k, the *projected* (target) inputs are:

$$x_{ik}^* = \theta_k^* \cdot x_{ik} - s_{ik}^{-*}$$

where s_{ik}^{-*} is the Phase II slack for input i.

7.2 Reduction metrics

$$\Delta x_{ik} = x_{ik} - x_{ik}^* \qquad \Delta x_{ik}^{\%} = \frac{\Delta x_{ik}}{x_{ik}} \times 100$$

These tell the fund manager: "to reach the frontier, you must reduce *operating expenses* by \$X (Y%) and *total assets deployed* by \$A (B%)."

7.3 Output file: `out/targets.csv`

Columns: `fund_id`, `theta_ccr`, then for each input: `target_*`, `reduction_abs_*`, `reduction_pct_*`.

8 Stage 2A: Simar–Wilson Bootstrap Bias Correction

8.1 Why bias correction is needed

The DEA estimator $\hat{\theta}_k$ is upward biased: the finite-sample frontier lies inside the true population frontier, inflating efficiency scores. The Simar–Wilson (1998) Algorithm 1 corrects this bias using a smoothed bootstrap.

8.2 Algorithm

Let $\hat{\boldsymbol{\theta}} = (\hat{\theta}_1, \ldots, \hat{\theta}_n)$ be the original DEA scores.

Step 1 — Silverman bandwidth.

$$h = 0.9 \cdot \min\left(\hat{\sigma},\ \frac{\mathrm{IQR}(\hat{\boldsymbol{\theta}})}{1.34}\right) \cdot n^{-1/5}$$

Step 2 — Reflected KDE resampling. For $b = 1, \ldots, B$:

a. Draw $\hat{\theta}_1^*, \ldots, \hat{\theta}_n^*$ i.i.d. from the kernel density of $\hat{\boldsymbol{\theta}}$ with bandwidth h.

b. Reflect values above 1 back: $\hat{\theta}_j^* \leftarrow 2 - \hat{\theta}_j^*$ if $\hat{\theta}_j^* > 1$.

c. Clip to $(0, 1]$.

Step 3 — Pseudo-dataset. Rescale inputs to impose pseudo-efficiency:

$$x_{ij}^{*(b)} = \frac{\hat{\theta}_j^{*(b)}}{\hat{\theta}_j} \cdot x_{ij}$$

Step 4 — Bootstrap DEA. Solve DEA on $(X^{*(b)}, Y)$ to get $\hat{\theta}_k^{*(b)}$.

Step 5 — Bias and corrected score.

$$\widehat{\mathrm{bias}}_k = \frac{1}{B}\sum_{b=1}^{B} \hat{\theta}_k^{*(b)} - \hat{\theta}_k \qquad \tilde{\theta}_k = \hat{\theta}_k - \widehat{\mathrm{bias}}_k$$

Step 6 — Confidence interval. The 95% CI is the $[2.5\%, 97.5\%]$ percentiles of $\{\hat{\theta}_k^{*(b)}\}_{b=1}^{B}$.

The default is $B = 2000$; $B = 200$ is used in CI/test runs. Bootstrap replications are parallelised with `joblib`.

8.3 Output file: `out/bias_corrected_scores.csv`

Columns: `fund_id`, `theta_raw`, `theta_bias_corrected`, `bias`, `ci_lower`, `ci_upper`.

9 Stage 2B: Random Forest Second Stage

9.1 Purpose

The second-stage regression identifies which environmental factors (inflation, exchange volatility, fund age) explain variation in bias-corrected efficiency scores $\tilde{\theta}_k$.

9.2 Model

$$\tilde{\theta}_k = f(\mathrm{inflation}_k,\ \mathrm{exchange_volatility}_k,\ \mathrm{fund_age}_k) + \varepsilon_k$$

$f(\cdot)$ is estimated by a Random Forest with:

- n_estimators $= 500$ trees
- max_features $= \sqrt{p}$ features per split
- Deterministic via `np.random.default_rng(seed)`

9.3 Feature importance measures

MDI importance (impurity-based):

$$I_j^{\text{MDI}} = \frac{1}{T}\sum_{t=1}^{T} \sum_{\text{node } v \in t:\, \text{split on } j} \frac{N_v}{N} \Delta\text{impurity}(v)$$

Permutation importance: Feature j is permuted 30 times; the mean decrease in R^2 is recorded. This avoids MDI's bias toward high-cardinality features.

9.4 Partial Dependence Plots

The marginal effect of feature j on the predicted score, averaging over all other features:

$$\hat{f}_j(x_j) = \frac{1}{n}\sum_{i=1}^{n} \hat{f}(x_j, \mathbf{x}_{i,-j})$$

Saved as `out/pdp_top3.png` for the three most important features.

9.5 Output files

- `out/rf_importance.csv` — feature name, MDI importance
- `out/pdp_top3.png` — partial dependence plots

10 Graph Descriptions

10.1 PCA Scree Plot

A bar chart with principal components on the x-axis and explained variance (%) on the y-axis. **What to look for:** if PC1 explains > 80%, the three inputs are nearly collinear (all scale with fund size). In that case, using PCA composites (`-use-pca-composite`) is recommended to avoid the "curse of dimensionality" in the DEA LP.

10.2 CCR vs. BCC Efficiency Bar Chart

Side-by-side bars per fund. BCC bars are always $\geq$ CCR bars. **What to look for:**

- Funds where CCR $\approx$ BCC: scale-efficient.
- Large gap (BCC $\gg$ CCR): scale inefficiency is the dominant problem.
- Low BCC: pure managerial inefficiency regardless of scale.

10.3 Partial Dependence Plots

Three sub-plots, one per top driver. The y-axis is the predicted $\tilde{\theta}$ and the x-axis is the feature value. A downward slope for *inflation* indicates that higher inflation reduces efficiency; an upward slope for *fund_age* suggests older funds are more efficient (institutional learning).

11 Process Verification: Python Test Snippets

The following snippets test the correctness of each pipeline stage. Run them with `uv run pytest explanation/test_process.py -v` after placing `test_process.py` alongside this file, or run interactively by pasting into a REPL inside the project directory.

11.1 Stage 0: Data loading and schema

```python
import pandas as pd
from pathlib import Path

def test_dataset_schema():
    df = pd.read_csv("tests/dataset.csv")

    # Correct number of rows and unique funds
    assert len(df) == 100, f"Expected 100 rows, got {len(df)}"
    assert df["fund_id"].nunique() == 20, "Expected 20 unique funds"
    assert sorted(df["year"].unique()) == [2018, 2019, 2020, 2021, 2022]

    # No (fund_id, year) duplicates
    assert df.duplicated(["fund_id", "year"]).sum() == 0

    # Financial columns must be positive (except net_investment_income_usd)
    for col in ["total_assets_usd", "operating_expenses_usd",
                "equity_debt_usd", "member_contributions_usd"]:
        assert (df[col] > 0).all(), f"{col} has non-positive values"

    # Macro variables match expected values per year
    expected = {2018: 10.6, 2019: 255.3, 2020: 557.2, 2021: 98.5, 2022: 243.8}
    for year, inf in expected.items():
        vals = df.loc[df["year"] == year, "inflation"].unique()
        assert len(vals) == 1 and abs(vals[0] - inf) < 0.2, \
            f"Inflation mismatch for {year}: {vals}"

    print("PASS: dataset schema OK")

test_dataset_schema()
```

Listing 1: Test: CSV loads with correct schema and types

11.2 Stage 1: PCA variance explained

```python
import numpy as np
import pandas as pd
from pension_toolkit.data_io import load_csv, get_dea_matrices
```

```python
from pension_toolkit.pca_utils import run_pca

def test_pca_variance():
    df = load_csv("tests/dataset.csv")
    numeric_cols = df.select_dtypes("number").columns.tolist()
    df_agg = df.groupby("fund_id")[numeric_cols].mean().reset_index()

    INPUT_COLS = ["operating_expenses_usd", "total_assets_usd", "equity_debt_usd"]
    X = df_agg[INPUT_COLS].to_numpy(dtype=float)

    result = run_pca(X, n_components=3, feature_names=INPUT_COLS)

    # Three components should explain >= 95% of variance
    assert result.cumulative_variance[-1] >= 0.95, \
        f"PCA explains only {result.cumulative_variance[-1]:.2%} of variance"

    # First component dominates (collinearity test: EVR1 > 0.60)
    assert result.explained_variance_ratio[0] >= 0.60, \
        f"PC1 explains only {result.explained_variance_ratio[0]:.2%}"

    # Components must be finite
    assert np.isfinite(result.components).all()

    print(f"PASS: PC1={result.explained_variance_ratio[0]:.1%}, "
          f"Cumulative={result.cumulative_variance[-1]:.1%}")

test_pca_variance()
```

Listing 2: Test: PCA retains >= 95% variance in 3 components

11.3 Stage 2: CCR DEA correctness

```python
import numpy as np
import pandas as pd
from pension_toolkit.data_io import load_csv, get_dea_matrices
from pension_toolkit.dea_core import dea_ccr_input_oriented

def test_ccr_dea():
    df = load_csv("tests/dataset.csv")
    numeric_cols = df.select_dtypes("number").columns.tolist()
    df_agg = df.groupby("fund_id")[numeric_cols].mean().reset_index()

    INPUT_COLS  = ["operating_expenses_usd", "total_assets_usd", "equity_debt_usd"]
    OUTPUT_COLS = ["net_investment_income_usd", "member_contributions_usd"]
```

```python
    fund_ids = df_agg["fund_id"].tolist()
    X = df_agg[INPUT_COLS].to_numpy(dtype=float)
    Y = df_agg[OUTPUT_COLS].to_numpy(dtype=float)

    result = dea_ccr_input_oriented(X, Y, fund_ids)

    # All scores must be in (0, 1]
    assert (result.theta > 0).all(),  "Zero efficiency score detected"
    assert (result.theta <= 1 + 1e-6).all(), "Score exceeds 1.0"

    # At least one fully efficient fund (theta == 1)
    assert (result.theta >= 1 - 1e-4).any(), \
        "No fund is on the CCR frontier -- possible data problem"

    # Not all funds are efficient (otherwise DEA is trivial)
    assert (result.theta < 0.999).any(), "All funds score 1.0 -- no inefficiency detected"

    # Lambda matrix shape
    n = len(fund_ids)
    assert result.lambdas.shape == (n, n)
    assert result.slacks_in.shape == (n, len(INPUT_COLS))

    print(f"PASS: CCR mean={result.theta.mean():.4f}, "
          f"efficient={( result.theta >= 1-1e-4).sum()}/{n}")

test_ccr_dea()
```

Listing 3: and frontier funds score 1]Test: CCR scores are in (0,1] and frontier funds score 1

11.4 Stage 3: BCC $\geq$ CCR and Scale Efficiency in (0, 1]

```python
import numpy as np
import pandas as pd
from pension_toolkit.data_io import load_csv, get_dea_matrices
from pension_toolkit.dea_core import dea_ccr_input_oriented, dea_bcc_input_oriented
from pension_toolkit.scale import compute_scale_efficiency

def test_scale_efficiency():
    df = load_csv("tests/dataset.csv")
    numeric_cols = df.select_dtypes("number").columns.tolist()
    df_agg = df.groupby("fund_id")[numeric_cols].mean().reset_index()

    INPUT_COLS  = ["operating_expenses_usd", "total_assets_usd", "equity_debt_usd"]
    OUTPUT_COLS = ["net_investment_income_usd", "member_contributions_usd"]
```

```python
    fund_ids = df_agg["fund_id"].tolist()
    X = df_agg[INPUT_COLS].to_numpy(dtype=float)
    Y = df_agg[OUTPUT_COLS].to_numpy(dtype=float)

    ccr = dea_ccr_input_oriented(X, Y, fund_ids)
    bcc = dea_bcc_input_oriented(X, Y, fund_ids)

    # BCC >= CCR for all DMUs
    assert (bcc.theta >= ccr.theta - 1e-6).all(), "BCC < CCR for some DMU"

    scale = compute_scale_efficiency(ccr, bcc)

    # Scale efficiency in (0, 1]
    assert (scale.scale_efficiency > 0).all()
    assert (scale.scale_efficiency <= 1 + 1e-6).all()

    # RTS classification is one of IRS / DRS / CRS
    valid_rts = {"IRS", "DRS", "CRS"}
    assert all(r in valid_rts for r in scale.rts_class)

    # All three classes should appear in a realistic dataset of 20 funds
    present = set(scale.rts_class)
    assert len(present) >= 2, f"Only {present} RTS classes observed"

    print(f"PASS: mean SE={scale.scale_efficiency.mean():.4f}, "
          f"IRS={scale.rts_class.count('IRS')}, "
          f"DRS={scale.rts_class.count('DRS')}, "
          f"CRS={scale.rts_class.count('CRS')}")

test_scale_efficiency()
```

Listing 4: Test: BCC dominance and scale efficiency bounds

11.5 Stage 4: Input reduction targets are valid

```python
import pandas as pd

def test_targets():
    targets = pd.read_csv("out/targets.csv")
    ccr     = pd.read_csv("out/efficiency_ccr.csv")
    dataset = pd.read_csv("tests/dataset.csv")

    numeric = dataset.select_dtypes("number").columns.tolist()
    df_agg  = dataset.groupby("fund_id")[numeric].mean().reset_index()

    INPUT_COLS = ["operating_expenses_usd", "total_assets_usd", "equity_debt_usd"]
```

```python

    for col in INPUT_COLS:
        target_col = f"target_{col}"
        if target_col not in targets.columns:
            continue
        # Targets must be non-negative
        assert (targets[target_col] >= 0).all(), f"Negative target for {col}"
        # Targets must be <= actual (efficiency <= 1 means contraction only)
        actual = df_agg.set_index("fund_id")[col]
        for _, row in targets.iterrows():
            act = actual.loc[row["fund_id"]]
            assert row[target_col] <= act + 1e-3, \
                f"Target > actual for {row['fund_id']} / {col}"

    # Efficient funds (theta==1) should have zero reduction
    efficient = ccr[ccr["theta_ccr"] >= 1 - 1e-4]["fund_id"].tolist()
    for fid in efficient:
        row = targets[targets["fund_id"] == fid]
        for col in INPUT_COLS:
            pct_col = f"reduction_pct_{col}"
            if pct_col in row.columns:
                assert row[pct_col].values[0] < 1.0, \
                    f"Efficient fund {fid} has non-zero reduction for {col}"

    print(f"PASS: targets valid for all {len(targets)} funds")

test_targets()
```

Listing 5: Test: target inputs are non-negative and <= actual inputs

11.6 Stage 5: Bootstrap bias correction properties

```python
import numpy as np
import pandas as pd
from pension_toolkit.data_io import load_csv, get_dea_matrices
from pension_toolkit.dea_core import dea_ccr_input_oriented
from pension_toolkit.bootstrap import simar_wilson

def test_bootstrap():
    df = load_csv("tests/dataset.csv")
    numeric = df.select_dtypes("number").columns.tolist()
    df_agg = df.groupby("fund_id")[numeric].mean().reset_index()

    INPUT_COLS  = ["operating_expenses_usd", "total_assets_usd", "equity_debt_usd"]
    OUTPUT_COLS = ["net_investment_income_usd", "member_contributions_usd"]
```

```python
    fund_ids = df_agg["fund_id"].tolist()
    X = df_agg[INPUT_COLS].to_numpy(dtype=float)
    Y = df_agg[OUTPUT_COLS].to_numpy(dtype=float)

    def ccr_func(X_, Y_):
        return dea_ccr_input_oriented(X_, Y_, fund_ids)

    result = simar_wilson(ccr_func, X, Y, fund_ids=fund_ids, B=200, seed=42)

    # Bias-corrected scores are in [0, 1]
    assert (result.theta_bias_corrected >= 0).all()
    assert (result.theta_bias_corrected <= 1 + 1e-6).all()

    # CIs are ordered: lower <= upper
    assert (result.ci_lower <= result.ci_upper + 1e-9).all()

    # CIs bracket the bias-corrected score (in most cases)
    inside = ((result.theta_bias_corrected >= result.ci_lower - 1e-4) &
              (result.theta_bias_corrected <= result.ci_upper + 1e-4))
    assert inside.mean() >= 0.80, \
        f"Only {inside.mean():.0%} of BC scores inside their CIs"

    # Reproducibility
    r2 = simar_wilson(ccr_func, X, Y, fund_ids=fund_ids, B=200, seed=42)
    np.testing.assert_array_almost_equal(
        result.theta_bias_corrected, r2.theta_bias_corrected,
        decimal=6, err_msg="Bootstrap is not reproducible with same seed"
    )

    print(f"PASS: mean raw={result.theta_raw.mean():.4f}, "
          f"mean BC={result.theta_bias_corrected.mean():.4f}, "
          f"mean bias={result.bias.mean():.4f}")

test_bootstrap()
```

Listing 6: Test: bias-corrected scores are <= raw scores and CIs are valid

11.7 Stage 6: Random Forest second stage

```python
import numpy as np
import pandas as pd
from pension_toolkit.data_io import load_csv
from pension_toolkit.ml_stage import fit_rf, ENV_FEATURE_COLS

def test_rf_second_stage():
    df = load_csv("tests/dataset.csv")
```

```python
    numeric = df.select_dtypes("number").columns.tolist()
    df_agg = df.groupby("fund_id")[numeric].mean().reset_index()

    # Load bias-corrected scores from a prior CLI run
    try:
        bc = pd.read_csv("out/bias_corrected_scores.csv")
    except FileNotFoundError:
        print("SKIP: run CLI pipeline first to generate out/bias_corrected_scores.csv")
        return

    bc = bc.merge(df_agg[["fund_id"] + ENV_FEATURE_COLS], on="fund_id")
    env_features = bc[ENV_FEATURE_COLS].to_numpy(dtype=float)
    scores = bc["theta_bias_corrected"].to_numpy(dtype=float)

    result = fit_rf(scores, env_features,
                    feature_names=ENV_FEATURE_COLS, seed=42)

    # MDI importance sums to 1
    total_imp = result.feature_importance["importance"].sum()
    assert abs(total_imp - 1.0) < 1e-6, f"Importance sum = {total_imp}"

    # All feature names accounted for
    assert set(result.feature_importance["feature"]) == set(ENV_FEATURE_COLS)

    # top3 contains valid feature names
    assert all(f in ENV_FEATURE_COLS for f in result.top3_features)

    # Permutation importance has correct shape
    assert len(result.permutation_importance) == len(ENV_FEATURE_COLS)

    print(f"PASS: CV R2 mean={result.cv_r2_scores.mean():.4f}, "
          f"top driver={result.top3_features[0]}")

test_rf_second_stage()
```

Listing 7: Test: RF fits without error and importance sums to 1

11.8 Stage 7: End-to-end pipeline sanity check

```python
from pathlib import Path
import pandas as pd

EXPECTED_OUTPUTS = [
    ("out/efficiency_ccr.csv",        ["fund_id", "theta_ccr"]),
    ("out/efficiency_vrs.csv",        ["fund_id", "theta_bcc"]),
```

```python
    ("out/scale.csv",                    ["fund_id", "scale_efficiency",
                                          "rts_classification"]),
    ("out/targets.csv",                  ["fund_id", "theta_ccr"]),
    ("out/bias_corrected_scores.csv", ["fund_id", "theta_raw",
                                          "theta_bias_corrected",
                                          "ci_lower", "ci_upper"]),
    ("out/rf_importance.csv",            ["feature", "importance"]),
]

def test_pipeline_outputs():
    for csv_path, required_cols in EXPECTED_OUTPUTS:
        p = Path(csv_path)
        assert p.exists(), f"Missing output: {csv_path}"
        df = pd.read_csv(p)
        assert len(df) > 0, f"Empty file: {csv_path}"
        for col in required_cols:
            assert col in df.columns, f"Column '{col}' missing from {csv_path}"

    assert Path("out/report.pdf").exists(), "PDF report not generated"
    assert Path("out/pdp_top3.png").exists(), "PDP plot not generated"

    print("PASS: all output files present with required columns")

test_pipeline_outputs()
```

Listing 8: Test: all output files are produced and non-empty

12 Running the Full Pipeline

```bash
# 1. Enter project directory and sync environment
cd prototype/pension_efficiency_toolkit
uv sync

# 2. Run full analysis (use real dataset, 200 bootstrap replications for speed)
uv run python -m pension_toolkit.cli analyze \
    --input tests/dataset.csv \
    --out out/ \
    --bootstrap-B 200 \
    --seed 42

# 3. Run unit tests
uv run pytest -q

# 4. Run process-verification snippets
uv run python explanation/test_process.py

```

```shell
# 5. Launch dashboard
uv run streamlit run pension_toolkit/ui_streamlit.py
```

Listing 9: Shell commands to reproduce all results

13 Key References

1. Charnes, A., Cooper, W.W., Rhodes, E. (1978). Measuring the efficiency of decision making units. *European Journal of Operational Research*, 2(6), 429–444.
2. Banker, R.D., Charnes, A., Cooper, W.W. (1984). Some models for estimating technical and scale inefficiencies in data envelopment analysis. *Management Science*, 30(9), 1078–1092.
3. Simar, L., Wilson, P.W. (1998). Sensitivity analysis of efficiency scores: How to bootstrap in nonparametric frontier models. *Management Science*, 44(1), 49–61.
4. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
5. Insurance and Pensions Commission Zimbabwe (IPEC). Annual Reports 2018–2022.